

Alignment Practical Exercises

Benjamin Redelings

Table of Contents

[1. Exercise \(Setup, AliView, alignment "by eye"\)](#)

- [1.1. Download the alignment files we will use](#)
- [1.2. Open a FASTA file in Aliview](#)
- [1.3. Align the sequences "by eye"](#)
- [1.4. Try some different programs \(optional\)](#)

[2. Exercise \(MAFFT, HoT, MUSCLE\)](#)

- [2.1. Make a ~/bin directory \(for programs\) and add it to your PATH](#)
- [2.2. Install fasta-flip.pl and compare-alignments to ~/bin](#)
- [2.3. MAFFT and HoT and Seaview](#)
- [2.4. Alignment with MUSCLE \(v5\)](#)

[3. Exercise \(FSA, PRANK\)](#)

- [3.1. Alignment with FSA](#)
- [3.2. Alignment with PRANK](#)

[4. Aligners and Trees](#)

[5. BALi-Phy \(part 2\)](#)

- [5.1. Creating a script file](#)
- [5.2. Command line options](#)
- [5.3. Running an analysis](#)
- [5.4. Monitoring an analysis](#)
- [5.5. Interpreting parameter names](#)
- [5.6. Summarizing the results of a run](#)
- [5.7. Comparing to MAFFT, FSA, and PRANK](#)
- [5.8. Improving the analysis](#)
- [5.9. Codon models for the coding data](#)
- [5.10. Further exploration...](#)

This tutorial assumes a UNIX command line. On Linux or Mac this is built in; on Windows install Cygwin or WSL.

1. Exercise (Setup, AliView, alignment "by eye")

1.1. Download the alignment files we will use

```
% cd // go to your home directory
% pwd // where are we? = print working directory
% mkdir alignment_files // make a subdirectory
% cd ~/alignment_files // enter the subdirectory
% wget http://www.bali-phy.org/examples.tgz // Download the files - Linux
% curl -O http://www.bali-phy.org/examples.tgz // Download the files - Mac
<Alternatively, download the files using your browser>
% tar -zxvf examples.tgz // extract the compressed archive
% ls examples // look inside the new directory that was created
% less examples/ferns/cleaned.fasta // examine the downloaded files – type q to quit
% alignment-info examples/ferns/cleaned.fasta // this will only work if bali-phy is already installed
```

1.2. Open a FASTA file in Aliview

```
% cd ~/alignment_files/examples
% cat test1.fasta
```

```
% aliview test1.fasta
```

Open the sequences in `~/alignment/examples/test1.fasta` using Aliview. The simplest way is to open Aliview, and then find `test1.fasta` using the file finder. Alternatively, if Aliview is your PATH, then you can run it from the command line. You can use `+` and `-` to zoom in and out.

1.3. Align the sequences "by eye"

Now add gaps (-) to align the letters in the sequences:

- Use SPACE and BACKSPACE to add and remove gaps
- Do not use DELETE to remove letters

Which alignment (or alignments) is most likely? Why?

1.4. Try some different programs (optional)

```
% mafft test1.fasta > test1.mafft.fasta
% muscle -align test1.fasta -output test1.mafft.fasta
% fsa test1.fasta > test1.fsa.fasta
```

Look at all these alignments in aliview.

2. Exercise (MAFFT, HoT, MUSCLE)

2.1. Make a `~/bin` directory (for programs) and add it to your PATH

```
% cd
% pwd // where are we? = print working directory
% ls // look around
% ls bin // does it already exist?
% mkdir ~/bin
% ls bin // does it exist now?

% ls .bash_profile .profile .zprofile // do you have .profile or .bash_profile or .zprofile?
% nano .profile // edit the startup script
OR
% nano .bash_profile // edit the startup script
OR
% nano .zprofile // edit the startup script
<Add the line "PATH=~/bin:$PATH".>

% cat ~/.bash_profile // Check if your changes are there
% cat ~/.profile // Check if your changes are there
% cat ~/.zprofile // Check if your changes are there

<Log out and log back in.>
% echo $PATH // check that ~/bin is in PATH
% echo $PATH | sed 's:/\n/g' // easier to read!
```

2.2. Install `fasta-flip.pl` and `compare-alignments` to `~/bin`

```
% cd ~/alignment_files/examples

% chmod +x fasta-flip.pl // make the file executable
% cp fasta-flip.pl ~/bin // copy the file to ~/bin
% which fasta-flip.pl // check to make sure its in your PATH

% chmod +x compare-alignments // make the file executable
% cp compare-alignments ~/bin // copy the file to ~/bin
% which compare-alignments // check to make sure its in your PATH
```

2.3. MAFFT and HoT and Seaview

```
% cp ferns/cleaned.fasta ferns.fasta // make a copy of a file
```

```
% mafft --auto ferns.fasta > ferns-mafft.fasta // try a simple sequence alignment
% fasta-flip.pl ferns.fasta | mafft --auto - | fasta-flip.pl > ferns-mafft-flipped.fasta
% aliview ferns-mafft.fasta & aliview ferns-mafft-flipped.fasta & // take a look at the two good alignments
% compare-alignments ferns-mafft.fasta ferns-mafft-flipped.fasta > ferns-compare.html
% firefox ferns-compare.html // load in firefox, then shrink the font
```

2.4. Alignment with MUSCLE (v5)

Look at the time to perform different numbers of iterations with MAFFT:

```
% cd ~/alignment_files/examples
% muscle -align ferns.fasta -output ferns-muscle.fasta // compute a MUSCLE alignment
% aliview ferns-muscle.fasta & // compare with the MAFFT alignment
% alignment-info ferns-muscle.fasta | grep columns // How long is the alignment?
% compare-alignments ferns-mafft.fasta ferns-muscle.fasta > ferns-compare-muscle-mafft.html
% firefox ferns-compare-muscle-mafft.html
```

```
% muscle -align ferns.fasta -output ferns-muscle.efa -diversified -replicates 10
% muscle -efa_explode ferns-muscle.efa
% aliview abc.1 &
% aliview acb.2 &
```

```
% muscle -letterconf ferns-muscle.efa -ref ferns-muscle.fasta -output letterconf.afa -html ferns-muscle.html
% firefox ferns-muscle.html
```

3. Exercise (FSA, PRANK)

3.1. Alignment with FSA

Lets run FSA with no penalty for wrong homologies, like ProbCons and Muscle v5.

```
% time fsa --fast --gapfactor 0 ferns.fasta > ferns-fsa0.fasta // This is similar to probcons/muscle5 – few
gaps.
% alignment-info ferns-fsa0.fasta | grep columns
% aliview ferns-fsa0.fasta &
```

Let's run FSA with the standard penalty for wrong homologies:

```
% time fsa --fast ferns.fasta > ferns-fsa.fasta // This is the default. Some gaps.
% alignment-info ferns-fsa.fasta | grep columns
% aliview ferns-fsa.fasta &
```

How long is the probcons-like alignment, versus the FSA alignment?

3.2. Alignment with PRANK

Now let's run PRANK:

```
% time prank -d=fern.fasta +F -o=fern-prank
% cp fern-prank.best.fas fern-prank.fasta
% alignment-info fern-prank.fasta | grep columns
% aliview fern-prank.fasta &
```

How long is the PRANK alignment? How is the shape different from the other alignments?

4. Aligners and Trees

Lets try comparing some aligners and the trees that they produce.

```
% cd
% cd alignment_files/examples/
% cp 5S-rRNA/25.fasta 5S-25.fasta
```

Try mafft:

```
% mafft --auto 5S-25.fasta > 5S-25-mafft.fasta
% FastTree -nt 5S-25-mafft.fasta > 5S-25-mafft.tree
```

Try muscle:

```
% muscle -align 5S-25.fasta -output 5S-25-muscle.fasta
% FastTree -nt 5S-25-muscle.fasta > 5S-25-muscle.tree
```

Try prank:

```
% prank -d=5S-25.fasta +F -o=5S-25-prank
% cp 5S-25-prank.best.fas 5S-25-prank.fasta
% FastTree -nt 5S-25-prank.fasta > 5S-25-prank.tree
```

Try fsa:

```
% fsa --fast 5S-25.fasta > 5S-25-fsa.fasta
% FastTree -nt 5S-25-fsa.fasta > 5S-25-fsa.tree
```

Try bali-phy:

```
% bali-phy 5S-25.fasta -S "gtr +> Rates.free(n=4) +> Covarion.gt01" --name 5S-25-gtr-R4-gt01 &
% bali-phy 5S-25.fasta -S "gtr +> Rates.free(n=4) +> Covarion.gt01" --name 5S-25-gtr-R4-gt01 &
```

5. BAli-Phy (part 2)

This exercise offers a quick overview of bali-phy with an emphasis on concrete analysis of real datasets. Other documentation includes:

- The [Users Guide](#) is much more complete.
- The [Tutorial](#) is similar to this exercise, but covers more functionality.
- You can run `tool --help` to see command line options for `tool`.
- You can run `man tool` to see the manual page for `tool`.
- Manual pages for bali-phy and tools are also available [online](#).

5.1. Creating a script file

First, go to the `~/alignment_files/examples` directory and create a text file called `ferns-config1.txt` that looks like this:

```
# sequence data for 7 partitions
align = ferns.fasta:5-8      # exon 1
align = ferns.fasta:12-444   # intron 1
align = ferns.fasta:454-596  # exon 2
align = ferns.fasta:609-844  # intron 2
align = ferns.fasta:865-948  # exon 3
align = ferns.fasta:953-1312 # intron 3
align = ferns.fasta:1329-1393 # exon 4

smodel = gtr +> Rates.gamma +> inv # substitution model

name = ferns-intron-exon      # determines name for output directory
```

It should be possible to copy and paste this text into the nano editor when running nano. Note the instructions on the bottom of the terminal when running nano: `^X` means **Control-X**.

```
% cd ~/alignment_files/examples
% nano ferns-config1.txt
% cat ferns-config1.txt
% bali-phy -c ferns-config1.txt --test | less
```

5.2. Command line options

A script file can be turned into a command-line by replacing commands like `command = value` with `--command value`. The exception is that for `align=file` commands, you can just write the filename by itself:

```
% bali-phy ferns.fasta:{5-8,12-444,454-596,609-844,865-948,953-1312,1329-1393} --smodel 'gtr +> Rates.gamma +> inv' --name=ferns-intron-exon --test
```

However, with partitioned analyses the command-line can become kind of long. Raw command lines are more useful with a simple unpartitioned analysis:

```
% bali-phy ferns.fasta --test
```

You can see the options for bali-phy by running:

```
% bali-phy help
```

You can also get help on particular options or models with the help command:

```
% bali-phy help smodel // help on option
% bali-phy help gtr // help on model
```

5.3. Running an analysis

After you get the `--test` command to work, remove the `--test` option and start two copies of this analysis:

```
% bali-phy -c ferns-config1.txt &
% bali-phy -c ferns-config1.txt &
```

These should create directories called `ferns-intron-exon-1` and `ferns-intron-exon-2`. Lets look inside one of these directories:

```
% ls ferns-intron-exon-1
% less ferns-intron-exon-1/C1.log // numerical parameter samples
% less ferns-intron-exon-1/C1.trees // tree samples
% less ferns-intron-exon-1/C1.P1.fastas // alignment samples for partition 1
% less ferns-intron-exon-1/C1.P2.fastas // alignment samples for partition 2
```

5.4. Monitoring an analysis

You can monitor an analysis in two ways. One way is to use the program **tracer**.

```
% tracer ferns-intron-exon-1/C1.log ferns-intron-exon-2/C1.log // you may need to double-click on Tracer and load log files via the menu.
```

We would like to see that the likelihood and posterior probability have stabilized. Another method of monitoring is just to generate the summary report.

5.5. Interpreting parameter names

The parameter estimates have names that look like `context1/value` or `context1/model:parameter`. For example `S1/gtr:pi[A]` indicates

- S1** The 1st substitution model
- gtr** The GTR (general, time-reversible) nucleotide substitution model.
- pi** The nucleotide frequencies
- [A]** The frequency for A (adenine).

We can get information on what a parameter means by running **bal-phy help model**:

```
% bali-phy help gtr
```

For more information on parameter names, see [the section on Output / Field names](#) in the Users Guide.

5.6. Summarizing the results of a run

```
% bp-analyze ferns-intron-exon-1 ferns-intron-exon-2
% firefox Results/index.html
```

Let's concatenate the summmary estimates for each partition to recover an alignment for the whole gene region.

```
% alignment-cat Results/P{1..7}.max.fasta > ferns-bali-phy.fasta
```

Since bali-phy is still running, we can get information on its progress by rerunning **bp-analyze**.

5.7. Comparing to MAFFT, FSA, and PRANK

We can see how the different aligners affect the total alignment length:

```
% alignment-info ferns-bali-phy.fasta | grep columns
% alignment-info ferns-prank.fasta | grep columns
% alignment-info ferns-fsa.fasta | grep columns
% alignment-info ferns-fsa0.fasta | grep columns
% alignment-info ferns-mafft.fasta | grep columns
```

Let's visually compare the alignment by loading all the alignments in aliview. Shrink each alignment so that it is very small, and look at the intron between exons 2 and 3.

```
% aliview ferns-bali-phy.fasta &
% aliview ferns-prank.fasta &
% aliview ferns-fsa.fasta &
% aliview ferns-fsa0.fasta &
% aliview ferns-mafft.fasta &
```

5.8. Improving the analysis

Lets make a few changes and improvements to our analysis.

- Lets share a single substitution model between all exon partitions. This means that all the exons share a single set of nucleotide frequencies, etc. We share substitution models between partitions by specifying the list of partitions that a substitution model applies to.
- Let's also share a single substitution model between all introns.
- Lets use the "free rates" model instead of site-rate heterogeneity. This means that the rate for each bin is allowed to vary independently. So we drop the "+> inv" part of the model
- Lets fix the alignment for the exons. We can do this by specifying an insertion-deletion model of "none" for those partitions.

Let's put the following into a configuration file called `ferns-config2.txt`.

```
# sequence data for 7 partitions
align = ferns.fasta:5-8      # exon 1
align = ferns.fasta:12-444   # intron 1
align = ferns.fasta:454-596  # exon 2
align = ferns.fasta:609-844  # intron 2
align = ferns.fasta:865-948  # exon 3
align = ferns.fasta:953-1312 # intron 3
align = ferns.fasta:1329-1393 # exon 4

smodel = 1,3,5,7:gtr +> Rates.free # substitution model
smodel = 2,4,6:gtr +> Rates.free   # substitution model

imodel = 1,3,5,7:none           # insertion-deletion model
imodel = 2,4,6:rs07             # insertion-deletion model

name = ferns-intron-exon2       # determines name for output directory
```

```
% bali-phy -c ferns-config2.txt &
% bali-phy -c ferns-config2.txt &
<wait a few minutes
% wc -l ferns-intron-exon2-*/C1.log           // How many iterations have been completed so far?
% bp-analyze ferns-intron-exon2-1 ferns-intron-exon2-2 // Generate a summary of completed iterations.
<wait a few more minutes
% wc -l ferns-intron-exon2-*/C1.log           // How many iterations have been completed so far?
% bp-analyze ferns-intron-exon2-1 ferns-intron-exon2-2 // Generate a summary of completed iterations.
% firefox Results/index.html &
```

5.9. Codon models for the coding data

Lets concatenate the exons to form the codon data. In order to load nucleotide sequences as codons, three conditions must be met:

1. **Sequence lengths must be multiples of three.**

AAA --- TTT is OK

AAA --- T-- is not OK.

2. Gaps must be codon-aligned.

AAA --- TTT is OK

AA- --A TT- is not OK.

3. The sequences must be in-frame and not have stop codons.

ATG AAT AAA is OK, because it translates to MNK.

AAT GAA TAA is not OK, because it translates to NE*, where* is a stop codon.

Note that spaces in FASTA files are allowed, but not required.

Let's extract the coding data:

```
% alignment-cat -c6-8,454-596,865-948,1329-1389 ferns.fasta --strip-gaps > coding.fasta
% alignment-translate coding.fasta | less           // check that the coding frame is correct
```

Now we can put the following into a configuration file called `ferns-config3.txt`.

```
# sequence data for 7 partitions
align = coding.fasta
align = ferns.fasta:12-444    # intron 1
align = ferns.fasta:609-844  # intron 2
align = ferns.fasta:953-1312 # intron 3

smodel = 1:gy94(pi=f1x4)
#smodel = 1:m3
#smodel = 1:gtr + > x3 +> dNdS +> mut_sel_aa
#smodel = 1:function(w:gtr +> x3 +> dNdS(omega=w) +> mut_sel_aa) +> m3
smodel = 2,3,4:gtr +> Rates.free

imodel = 1:none
imodel = 2,3,4:rs07

name = ferns-intron-exon3
```

```
% bali-phy -c ferns-config3.txt &
% bali-phy -c ferns-config3.txt &
% wc -l ferns-intron-exon3-*/C1.log           // How many iterations have been completed so far?
% bp-analyze ferns-intron-exon3-1 ferns-intron-exon3-2 // Generate a summary of completed iterations.
<wait a few more minutes
% wc -l ferns-intron-exon3-*/C1.log           // How many iterations have been completed so far?
% bp-analyze ferns-intron-exon3-1 ferns-intron-exon3-2 // Generate a summary of completed iterations.
<wait a few minutes
% firefox Results/index.html &
```

5.10. Further exploration...

For more information see:

- The [Tutorial](#) is similar to this exercise, but covers more functionality.
- The [Users Guide](#) additionally covers ancestral sequence reconstruction, setting priors, etc.